

ШАБЛОННЫЕ КЛАССЫ

Семинар 15

Шаблон класса

Создание шаблона класса аналогично созданию шаблона функции.

```
template <class T>  
class Array  
{  
    // ...  
};
```

Функции-члены шаблонного класса должны быть определены в заголовочном файле.

Шаблон класса

array.h

```

4. #include <stdexcept>
5.
6. template <class T>
7. class Array
8. {
9. private:
10.     int m_length;
11.     T *m_data;
12.
13. public:
14.     Array()
15.     {
16.         m_length = 0;
17.         m_data = nullptr;
18.     }
19.     Array(int length)
20.     {
21.         m_data = new T[length];
22.         m_length = length;
23.     }
24.     ~Array()
25.     {
26.         delete[] m_data;
27.     }
28.     T& operator[](int index)
29.     {
30.         if (index < 0 || index > m_length)
31.             throw std::out_of_range("array error");
32.         return m_data[index];
33.     }
34.     void Erase();
35.     // Длина массива не зависит от типа элементов массива
36.     int getLength();
37. };

```

```

39. // метод, определенный вне тела класса,
40. // нуждается в собственном определении шаблона метода
41. template <typename T>
42. int Array<T>::getLength()
43. {
44.     return m_length;
45. }
46. template <typename T>
47. void Array<T>::Erase()
48. {
49.     delete[] m_data;
50.     // чтобы на выходе не получить висячий указатель
51.     m_data = nullptr;
52.     m_length = 0;
53. }

```

Шаблон класса

array.h

```

4 #include <stdexcept>
5
6 template <class T>
7 class Array
8 {
9 private:
10     int m_length;
11     T *m_data;
12
13 public:
14     Array()
15     {
16         m_length = 0;
17         m_data = nullptr;
18     }
19     Array(int length)
20     {
21         m_data = new T[length];
22         m_length = length;
23     }
24     ~Array()
25     {
26         delete[] m_data;
27     }
28     T& operator[](int index)
29     {
30         if (index < 0 || index > m_length)
31             throw std::out_of_range("array error");
32         return m_data[index];
33     }
34     void Erase();
35     // Длина массива не зависит от типа элементов массива
36     int getlength();
37 };

```

main.cpp

```

1 #include <iostream>
2 #include "array.h"
3
4 int main()
5 {
6     Array<int> intArray(10);
7     Array<double> doubleArray(10);
8
9     for (int count = 0; count < intArray.getLength(); ++count)
10     {
11         intArray[count] = count;
12         doubleArray[count] = count + 0.5;
13     }
14
15     for (int count = intArray.getLength() - 1; count >= 0; --count)
16         std::cout << intArray[count] << "\t" << doubleArray[count] << "\n";
17
18     return 0;
19 }

```

```

chernausov@DESKTOP-Igor:~/ex$ ./a.out
9      9.5
8      8.5
7      7.5
6      6.5
5      5.5
4      4.5
3      3.5
2      2.5
1      1.5
0      0.5

```

Шаблоны классов

Шаблоны классов работают точно так же, как и шаблоны функций:

- компилятор копирует шаблон класса, заменяя типы параметров шаблона класса на фактические типы данных, а затем компилирует эту копию.

Если есть шаблон класса и он не используется – компилятор не будет его компилировать.

Например, структуры данных из STL (`std::vector`, `std::queue` и др.) – шаблонные классы.

Шаблоны классов и заголовочные файлы

Шаблон не является ни классом, ни функцией — это трафарет, используемый для создания классов или функций.

Для array из предыдущего примера создадим .h и .cpp файлы:

```

1 //Header Array.h
2 #ifndef array_h
3 #define array_h
4
5 #include <iostream>
6
7 template <typename T>
8 class Array
9 {
10 private:
11     int m_length;
12     T* m_data;
13
14 public:
15     Array()
16     {
17         m_length = 0;
18         m_data = nullptr;
19     }
20     Array(int length)
21     {
22         m_data = new T[length];
23         m_length = length;
24     }
25     ~Array()
26     {
27         delete[] m_data;
28     }
29     // operator[] (int index)
30     {
31         if (index < 0 || index > m_length)
32             throw std::out_of_range("array error");
33         return m_data[index];
34     }
35     void Erase()
36     {
37         // delete m_data;
38         // delete m_data;
39         // delete m_data;
40     }
41 };
42
43 #endif

```

```

1 #include "new_array.h"
2
3 // метод, определенный вне тела класса,
4 // нуждается в собственном определении шаблона метода
5 template <typename T>
6 int Array<T>::getLength()
7 {
8     return m_length;
9 }
10 template <typename T>
11 void Array<T>::Erase()
12 {
13     delete[] m_data;
14     // чтобы на выходе не получить висячий указатель
15     m_data = nullptr;
16     m_length = 0;
17 }

```

При компиляции получим следующие ошибки:

```

chernousov@DESKTOP-Igor:~/ex$ g++ new_main_array.cpp new_array.cpp
/usr/bin/ld: /tmp/ccSyljmw.o: in function 'main':
new_main_array.cpp:(.text+0x4d): undefined reference to 'Array<int>::getLength()'
/usr/bin/ld: new_main_array.cpp:(.text+0xab): undefined reference to 'Array<int>::getLength()'
collect2: error: ld returned 1 exit status

```

Параметр non-type

Параметр non-type в шаблоне – специальный параметр шаблона, который заменяется не типом данных, а конкретным значением.

Этим значением может быть:

- целочисленное значение или перечисление;
- указатель или ссылка на объект класса;
- указатель или ссылка на функцию;
- указатель или ссылка на метод класса;
- `std::nullptr_t`.

```
1 // size является параметром non-type в шаблоне класса
2 template <class T, int size>
3 class StaticArray
4 {
5     private:
6         // Параметр non-type - размер выделяемого массива
7         T m_array[size];
8
9     public:
10        T* getArray();
11
12        T& operator[](int index)
13        {
14            return m_array[index];
15        }
16    };
```

Параметры шаблонов по умолчанию

Шаблонные классы и функции могут иметь типы по умолчанию. Если формальный параметр шаблона имеет значение по умолчанию, то все следующие за ним параметры также должны иметь значения по умолчанию.

```
template <unsigned int N, typename elem_t = uint32_t>  
struct TSetNBit  
{  
    // ...  
};
```


Частичная специализация

Если у шаблона класса есть несколько параметров, то можно специализировать его только для одного или нескольких аргументов, оставляя другие неспециализированными.

Если для шаблона класса объявлены частичные специализации, компилятор выбирает то определение, которое является наиболее специализированным для заданных аргументов. Если же ни одно из них не подходит, используется общее определение шаблона.

Частичная специализация

```

1  template<typename Type, int Size>
2  class SimpleStaticArray
3  {
4  public:
5      SimpleStaticArray()
6          : elements()
7      {
8      }
9
10     int getNum() const
11     {
12         return Size;
13     }
14
15     Type getElement(int inIndex) const
16     {
17         return elements[inIndex];
18     }
19
20     void setElement(int inIndex, Type inValue)
21     {
22         elements[inIndex] = inValue;
23     }
24
25 private:
26     Type elements[Size];
27 };

```

```

1  template<int Size>
2  class SimpleStaticArray<bool, Size>
3  {
4  public:
5      SimpleStaticArray()
6          : elementsMemory()
7      {
8      }
9
10     int getNum() const {
11         return Size;
12     }
13
14     bool getElement(int inIndex) const {
15         const BitArrayAccessData accessData = getAccessData(inIndex);
16         const unsigned char elementMask = (1 << accessData.bitIndexInByte);
17         return elementsMemory[accessData.byteIndex] & elementMask;
18     }
19
20     void setElement(int inIndex, bool inValue) {
21         const BitArrayAccessData accessData = getAccessData(inIndex);
22
23         const unsigned char elementMask = (1 << accessData.bitIndexInByte);
24         elementsMemory[accessData.byteIndex] =
25             (elementsMemory[accessData.byteIndex] & ~elementMask) |
26             (inValue ? elementMask : 0);
27     }
28
29 private:
30     static BitArrayAccessData getAccessData(int inElementIndex)
31     {
32         BitArrayAccessData result;
33         result.byteIndex = inElementIndex / 8;
34         result.bitIndexInByte = inElementIndex - result.byteIndex * 8;
35         return result;
36     },
37     unsigned char elementsMemory[Size / (sizeof(unsigned char) * 8) + 1];
38 };

```

Подробнее

1. https://en.cppreference.com/w/cpp/language/class_template
2. https://habr.com/ru/articles/599801/#_9_PartialSpecializations
3. <http://cpp-reference.ru/articles/expression-templates/>

Задачи

1. Реализовать шаблонный класс `Pair`, который позволяет передавать данные одного типа парами.
Реализовать функции получения первого и второго значения из пары.
2. Реализовать шаблонный класс для работы с квадратными матрицами. В качестве параметров шаблона передавать тип значения в матрице и ее размер.
Перегрузить операторы сложения, вычитания, умножения и вывода в поток